

Ejercicio Evaluable 1: Colas de mensajes

Sistemas Distribuidos 2025/26

Alicia Mei García Morín → 100498828
Ana Grima Vázquez de Prada → 100495785

1. Parte A: Diseño de la estructura de datos y sincronización

1.1. Almacenamiento local

Para la gestión de las tuplas $\langle key-value1-value2-value3 \rangle$, se ha implementado en *claves.c* una lista enlazada dinámica. Esta decisión simplifica la gestión de memoria (malloc/free por nodo) y evita preasignar capacidad, cumpliendo con el requisito de no imponer un límite fijo en el número de elementos que se pueden almacenar. Cada elemento de la lista se define mediante una estructura llamada *Nodo*, que contiene:

- *key*: una cadena de caracteres de hasta 255 caracteres útiles (256 en total con el código fin de cadena).
- *value1*: una cadena de caracteres de longitud similar a la clave.
- *N_value2*: un entero que indica la dimensión de un vector de floats (entre 1 y 32).
- *V_value2*: un vector de tipo float para almacenar hasta 32 elementos.
- *value3*: una estructura *Paquete* con tres enteros (x, y, z).
- *next*: un puntero al siguiente nodo de la lista.

El sistema utiliza un puntero global estático *g_head* que apunta al inicio de la lista, facilitando operaciones de inserción al principio de la misma para mayor eficiencia. La inserción al principio es $O(1)$ y el orden de almacenamiento es irrelevante porque la API no expone ningún recorrido ordenado de claves. Las operaciones que requieren localizar una clave (*get_value*, *modify_value*, *delete_key* y *exist*) implican recorrer la lista secuencialmente, por lo que tienen un coste $O(n)$.

1.2. Prevención de condiciones de carrera

Tal como exige el enunciado, todas las operaciones de la API deben realizarse de forma atómica. Para garantizar la consistencia de la lista enlazada en entornos concurrentes, se define una sección crítica que protege toda la estructura de datos compartida usando un mutex global de POSIX (*pthread_mutex_t*), inicializado de forma estática como *g_mutex*.

El mecanismo de sincronización sigue este patrón en todas las funciones (*set_value*, *get_value*, *modify_value*, *delete_key*, *exist* y *destroy*):

1. Bloqueo: se invoca *pthread_mutex_lock(&g_mutex)* al inicio de la función.
2. Operación: se accede o modifica la lista de forma segura.
3. Desbloqueo: se invoca *pthread_mutex_unlock(&g_mutex)* antes de retornar el control, asegurando que el mutex se libere incluso en caso de error prematuro (como cuando una clave no existe).

Como decisión de diseño, las comprobaciones de error al liberar el mutex (*unlock*) se gestionan priorizando la coherencia funcional de cara al cliente, evitando reportar un error de inserción o lectura si la operación sobre la estructura de datos ya se había completado con éxito. En cambio, en *destroy()* se comprueba el desbloqueo de forma más estricta porque deja el sistema en un estado “reset” y conviene señalar cualquier anomalía interna.

2. Parte B: Arquitectura distribuida y concurrencia

2.1. Modelo de concurrencia en el servidor

Decidimos crear un modelo de servidor concurrente basado en hilos bajo demanda para procesar múltiples peticiones de clientes simultáneamente. Lo preferimos así para evitar tener que gestionar un buffer intermedio de peticiones y sincronización adicional entre dispatcher y pool, ya que nos pareció más simple y que encajaba bien. El funcionamiento se divide en dos roles principales dentro del proceso *servidor-mq.c*:

- Hilo principal (dispatcher): se encarga de inicializar la cola de mensajes del servidor */mq_claves_srv_100495785* y entra en un bucle infinito de escucha. Cada vez que recibe un mensaje a través de *mq_receive*, reserva memoria dinámica para los argumentos de la petición y crea un hilo nuevo utilizando *pthread_create* para gestionar dicha solicitud específica.
- Hilos trabajadores (workers): cada hilo ejecuta la función *worker_fn*, que extrae los datos de la estructura *request_t* (cada worker trabaja sobre una copia privada de *request_t*) e invoca la función correspondiente de la biblioteca *libclaves.so*, garantizando atomicidad gracias al mutex interno de la biblioteca. Y envía la respuesta al cliente a través de su cola específica. Luego, liberada la memoria del argumento, el hilo finaliza. Los hilos se crean en modo *detached*, para que el sistema libere automáticamente sus recursos al finalizar, evitando la necesidad de realizar *pthread_join* y simplificando la gestión del ciclo de vida de los hilos en el servidor.

2.2. Protocolo de comunicación en el servidor

La comunicación entre el cliente, a través del *proxy-mq.c*, y el servidor se basa exclusivamente en colas de mensajes POSIX siguiendo un esquema de petición-respuesta. Se han definido dos estructuras de datos principales para el intercambio de información:

- Estructura de solicitud (*request_t*): enviada por el cliente al servidor. Contiene el tipo de operación *op_t*, el nombre de una cola de respuesta única generada por el cliente y todos los parámetros necesarios para la operación (clave, valor1, vectores, etc.).
- Estructura de respuesta (*response_t*): enviada por el servidor al cliente. Contiene el código de retorno de la operación y, en el caso de *get_value*, los campos necesarios para devolver los datos asociados a la clave.

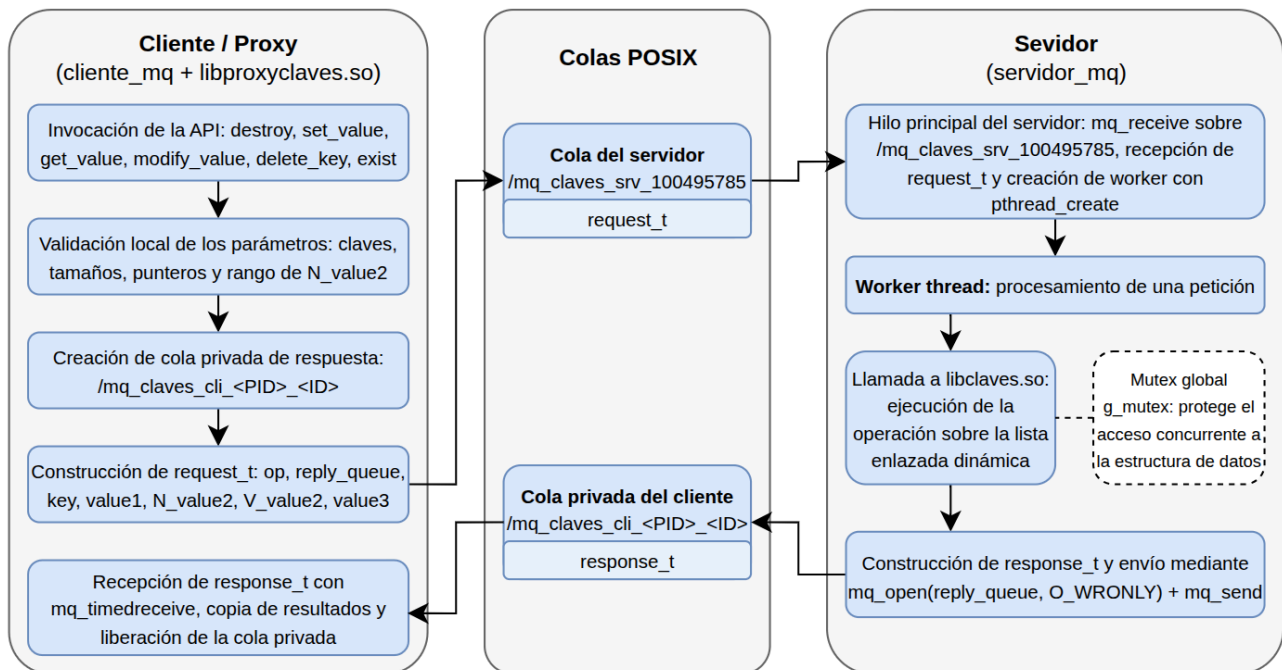
El protocolo sigue estos pasos:

1. El cliente genera un nombre de cola único, basado en su PID y un contador.
2. El cliente envía la estructura *request_t* a la cola pública del servidor */mq_claves_srv_100495785*.
3. El cliente queda bloqueado en su cola privada esperando la *response_t* del servidor. El uso de una cola de respuesta privada por cliente evita interferencias entre respuestas concurrentes de diferentes peticiones. De esta forma, el servidor puede procesar solicitudes en paralelo sin necesidad de mantener estado adicional para identificar al cliente que originó cada petición.
4. Tras recibir la respuesta o expirar un timeout, el cliente cierra y elimina su cola temporal para no dejar recursos huérfanos en */dev/mqueue*. El uso de *mq_timedreceive* con timeout permite detectar posibles fallos de comunicación (por ejemplo, si el servidor deja de responder). En ese caso, el

proxy cliente traduce esta situación en un código de error de comunicaciones, evitando bloqueos indefinidos en el cliente.

2.3. Diagrama de flujo

Protocolo de comunicación cliente-servidor basado en colas POSIX para esta práctica:



3. Batería de pruebas y tratamiento de errores

3.1. Gestión de errores

El diseño contempla una gestión de errores robusta basada en los valores de retorno definidos en la interfaz *claves.h*. Estas comprobaciones se realizan antes de acceder a la estructura de datos o de enviar solicitudes al servidor, siguiendo un enfoque de validación temprana que evita operaciones innecesarias y simplifica el control de errores. Los criterios de error aplicados incluyen:

- Validación de rangos: se devuelve -1 si *N_value2* es menor que 1 o mayor que 32.
- Validación de strings: se verifica que las claves y el valor *value1* no superen los 255 caracteres.
- Punteros nulos: el paso de valores *NULL* en parámetros obligatorios (como *key* o vectores) da lugar a un error.
- Estado del servicio: intentar insertar una clave que ya existe o modificar/borrar una que no existe devuelve -1.
- Errores internos: fallos en la reserva de memoria (*malloc*) o problemas al bloquear el mutex también se comunican con -1.

En la versión distribuida (Parte B) se distingue además entre errores propios del servicio y errores de comunicación. Mientras que los errores lógicos del servicio continúan devolviendo -1, los fallos en el intercambio de mensajes (timeout, imposibilidad de abrir colas, etc.) se traducen en el valor -2 desde el proxy cliente.

3.2. Plan de pruebas

La batería de pruebas implementada en *app-cliente.c* verifica cada servicio de la biblioteca *libclaves.so* de forma automatizada. Para ello, se han desarrollado macros de aserción (*CHECK_EQ_INT* y *CHECK_EQ_STR*) que validan automáticamente los códigos de retorno y la integridad de los datos obtenidos frente a los resultados esperados, notificando por consola el éxito ([OK]) o el fallo ([FAIL]) de cada test. Además, para permitir la ejecución de múltiples clientes concurrentes sin interferencias entre ellos, las claves utilizadas en las pruebas se generan dinámicamente incorporando el identificador del proceso (*PID*) mediante *getpid()*. De esta forma, cada cliente opera sobre su propio conjunto de claves, evitando colisiones cuando varios clientes ejecutan la batería de pruebas simultáneamente. El plan de pruebas incluye:

- Estado inicial: verificación de que las claves generadas dinámicamente no existen previamente mediante llamadas a *exist()* y *get_value()*.
- Operaciones básicas: inserción, consulta, modificación y borrado de tuplas con datos válidos, verificando la integridad de cada campo (*value1*, vector de float y struct Paquete).
- Casos límite de N: pruebas con *N_value2* = 1 y *N_value2* = 32 para asegurar que los límites del vector se gestionen correctamente.
- Pruebas negativas:
 - Inserción de claves duplicadas.
 - Uso de valores de *N* fuera de rango (0 y 33).
 - Envío de cadenas excesivamente largas (>255 caracteres).
 - Llamadas con punteros *NULL*.
- Persistencia y consistencia: verificación de que tras borrar una clave, esta ya no es accesible y tras una modificación, los datos antiguos han sido reemplazados correctamente.

4. Manual de compilación y ejecución

4.1. Uso del Makefile

1. **Compilación de la biblioteca local (*libclaves.so*):** el fichero *claves.c* se compila como biblioteca dinámica utilizando código independiente de posición (*-fPIC*).
2. **Compilación del cliente local (*cliente_local*):** *app-cliente.c* se enlaza con la biblioteca *libclaves.so*. Este ejecutable permite comprobar el funcionamiento de la API directamente sobre la implementación local.
3. **Compilación del proxy cliente (*libproxyclaves.so*):** en la parte distribuida se implementa un stub cliente (*proxy-mq.c*) que encapsula la comunicación con el servidor mediante colas POSIX. Este componente también se compila como biblioteca dinámica para mantener la misma interfaz que la API original.
4. **Compilación del cliente distribuido (*cliente_mq*):** se reutiliza el mismo programa de pruebas *app-cliente.c*, pero enlazándolo con *libproxyclaves.so* en lugar de *libclaves.so*. De este modo se verifica que la API mantiene el mismo comportamiento independientemente de si el acceso es local o remoto.
5. **Compilación del servidor (*servidor_mq*):** el servidor (*servidor-mq.c*) se compila como ejecutable independiente y se enlaza con *libclaves.so*, reutilizando la implementación local del servicio para procesar las operaciones solicitadas por los clientes.

Con *make* se generan todos los ficheros mencionados de la Parte A y la Parte B. Además, el Makefile incluye *make clean*, que elimina los ficheros generados de ambas partes y provoca una finalización controlada del servidor, liberando la cola de mensajes del sistema.

4.2. Ejecución

Prueba de la Parte A: El cliente local ejecuta la batería de pruebas directamente sobre la biblioteca libclaves.so con `./cliente_local`. Si todas las pruebas se ejecutan correctamente, el programa finaliza mostrando: `=== TODO OK ===`

Prueba de la Parte B: primero se ejecuta el servidor con `./servidor_mq` (crea la cola de mensajes `/mq_claves_srv_100495785` y queda a la espera de peticiones de los clientes). En otra terminal, ejecutamos el cliente con `./cliente_mq`. La batería de pruebas es la misma que en la versión local, por lo que el resultado esperado también es: `=== TODO OK ===`. Además, se pueden ejecutar varios clientes de forma concurrente usando `for i in {1..x}; do ./cliente_mq & done; wait`, siendo x el número de clientes deseados. Para finalizar el servidor, se puede ejecutar `make clean`, que provoca la terminación del servidor en ejecución mostrando el mensaje “Cerrando el servidor de forma segura”.

La misma batería de pruebas se ejecuta tanto contra la implementación local (*cliente_local*) como contra la versión distribuida (*cliente_mq*), lo que permite verificar que el comportamiento externo del servicio se mantiene idéntico independientemente del mecanismo de comunicación utilizado.

5. Trabajo extra

Como trabajo extra, hicimos algunas pruebas de rendimiento para comparar el coste de la versión local y la versión distribuida del servicio. Para ello se midió el tiempo total de ejecución de la batería de pruebas usando el comando `time`.

En una terminal: `time ./cliente_local` y `./servidor_mq`. En otra terminal: `time ./cliente_mq`.

Como era esperable, la versión distribuida presenta un mayor tiempo de ejecución debido al coste adicional asociado a la comunicación entre procesos, el envío y recepción de mensajes mediante colas POSIX y la creación de hilos en el servidor para procesar cada solicitud.

También hicimos pruebas de concurrencia ejecutando distintos números de clientes simultáneamente para evaluar el comportamiento del servidor bajo carga. Para ello se lanzaron varios procesos *cliente_mq* en paralelo mediante ejecución en segundo plano. En todas las pruebas el sistema completó correctamente la batería de tests, lo que confirma que el servidor gestiona adecuadamente múltiples peticiones concurrentes. Como es esperable, al aumentar el número de clientes, el tiempo total de ejecución también aumenta debido al coste adicional. No obstante, los tiempos obtenidos se mantienen reducidos incluso con varios clientes simultáneos (por ejemplo, 0,073s para 20 clientes), lo que indica que la arquitectura concurrente implementada escala correctamente. Por motivos de espacio, no se incluyen todas las mediciones realizadas para distintos números de clientes.

Valores obtenidos:

cliente_local	<code>real 0m0,005s</code>
cliente_mq	<code>real 0m0,011s</code>
20 cliente_mq	<code>real 0m0,073s</code>

***Nota:**

- Se ha hecho uso de la IA (Github Copilot, ChatGPT) como herramienta de apoyo para la realización de la práctica, sin que en ningún momento ponga en duda nuestra autoría de esta (resolver dudas específicas, ayudar a la redacción de la memoria).